JECRC UNIVERSITY

BUILD YOUR WORLD

Department of Engineering

Session 2026-27

PROMPT ENGINEERING

(Course Code: BCO610A)

HW-06: RULES THAT ACTUALLY BITE

Cursor: AI Inside the Editor

Submitted By:

Name – Aditya Ranjan

Enrollment No. – 25BCON1294

Branch – B.Tech. CSE (AI&DS)

Semester – 3

Submitted To:

Mr. Ajay Sharma

Prompt Engineering

1. Activity A — Capture Reference + Explore Two Modes

Original Output Reference

The supplied messy script produces the following reference output:

```
REPORT

Rahul 258 86.0 B

Priya 205 68.33333333333333 D

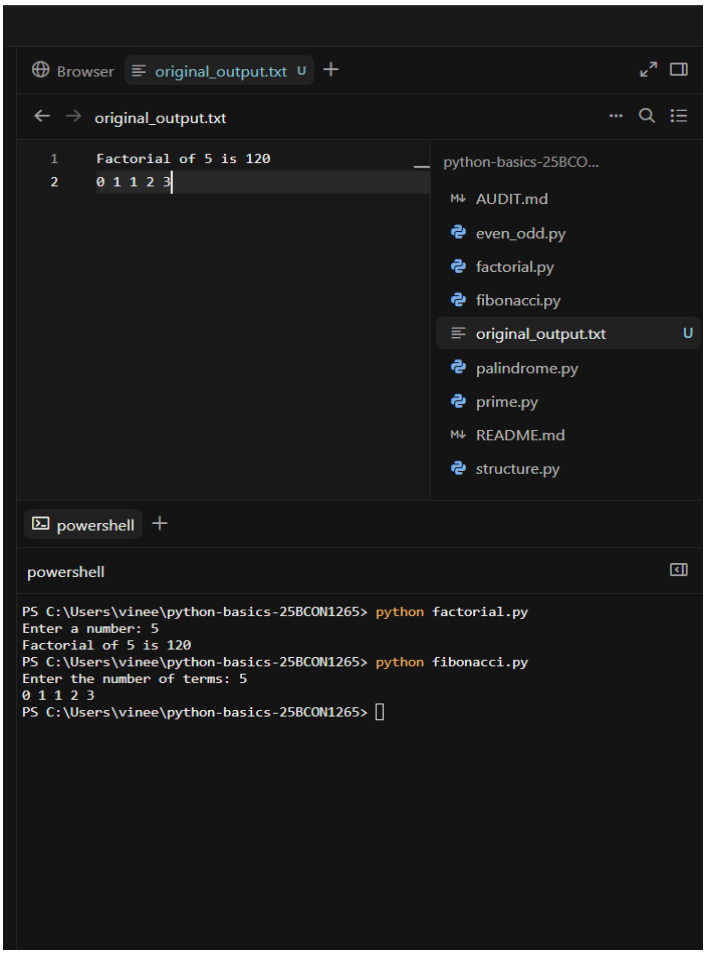
Amit 269 89.66666666666667 B

Sneha 173 57.666666666666664 F

Vikram 249 83.0 B

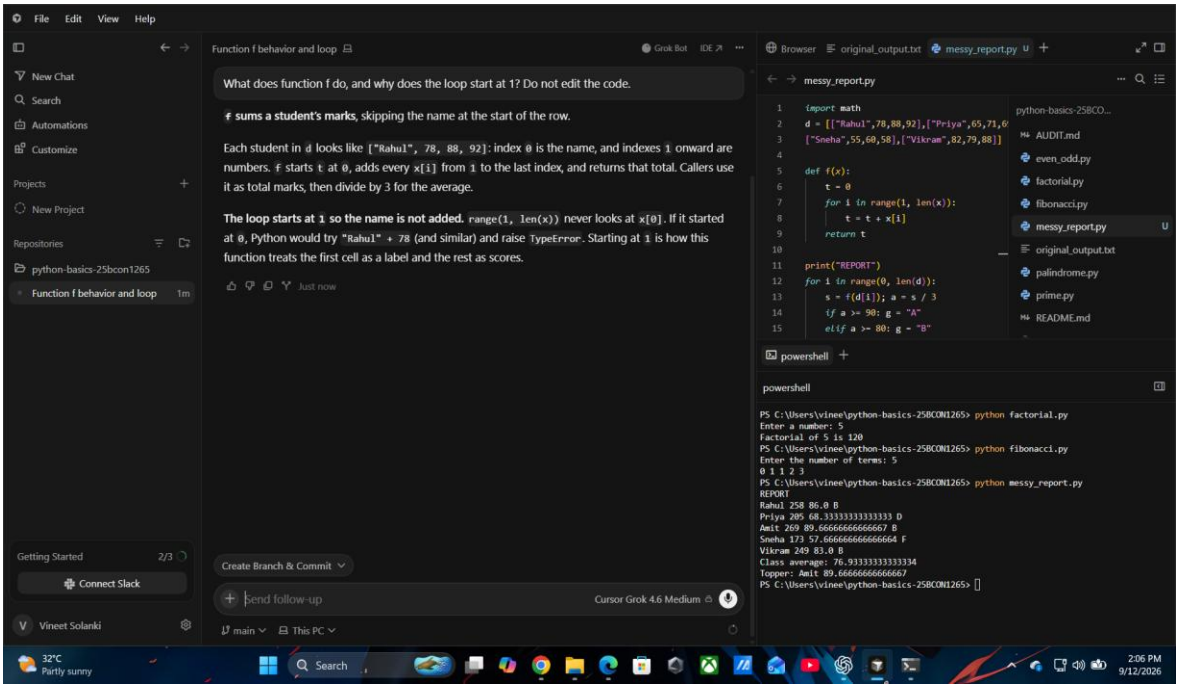
Class average: 76.93333333333334

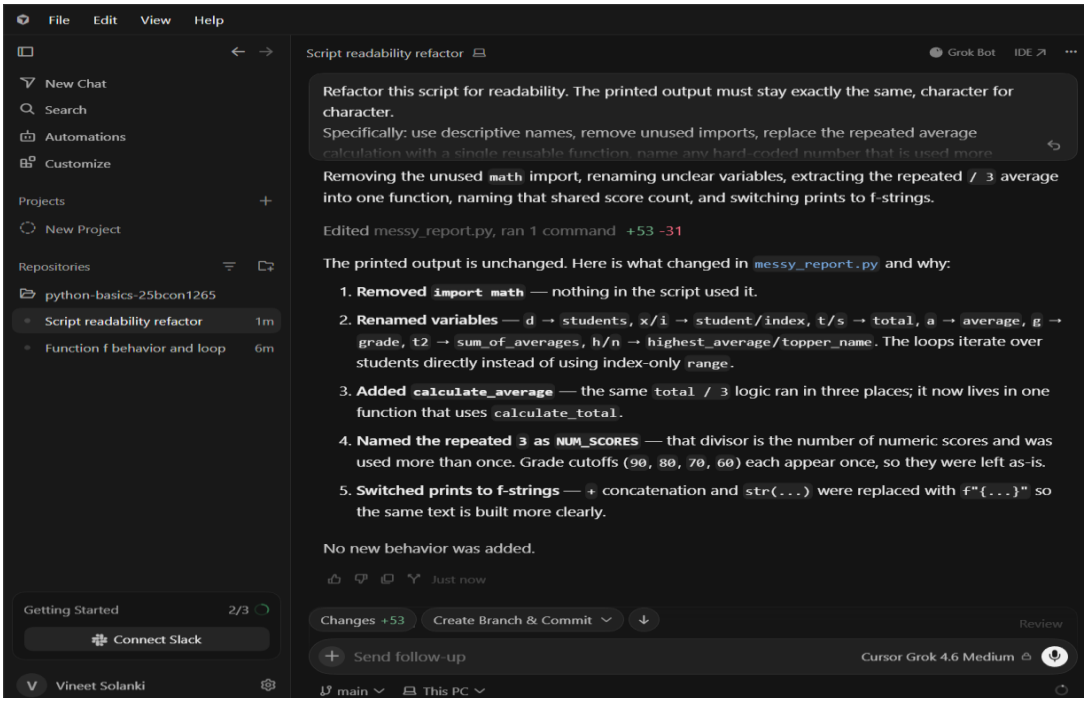
Topper: Amit 89.66666666666667
```



Two AI Modes

Mode	What I used it for	Observation
Understand-only	Ask what function f does and why the loop starts at 1.	Cursor explained the function correctly without changing the code. The loop starts at 1 to skip the student's name at index 0.
Inline edit	Rename f and add a one-line docstring; inspect the diff and rerun.	Cursor renamed the function and added the docstring. I reviewed the diff and confirmed the output stayed unchanged.





2. Activity B — Refactor and Verify

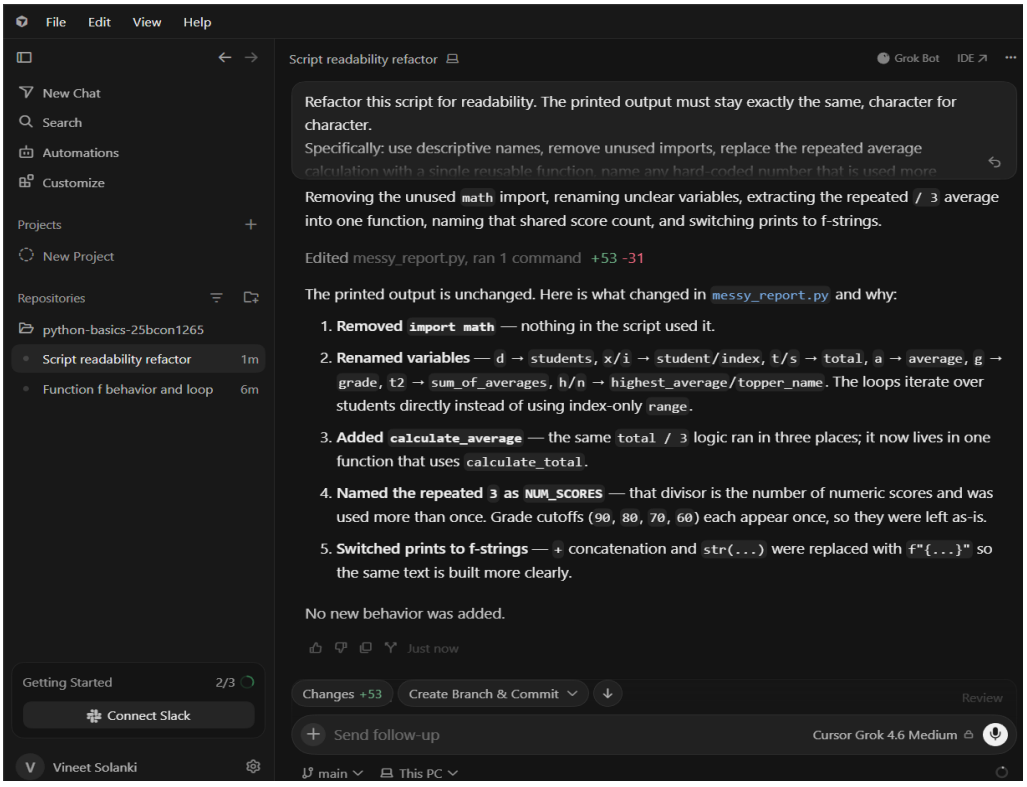
Refactor Prompt Used

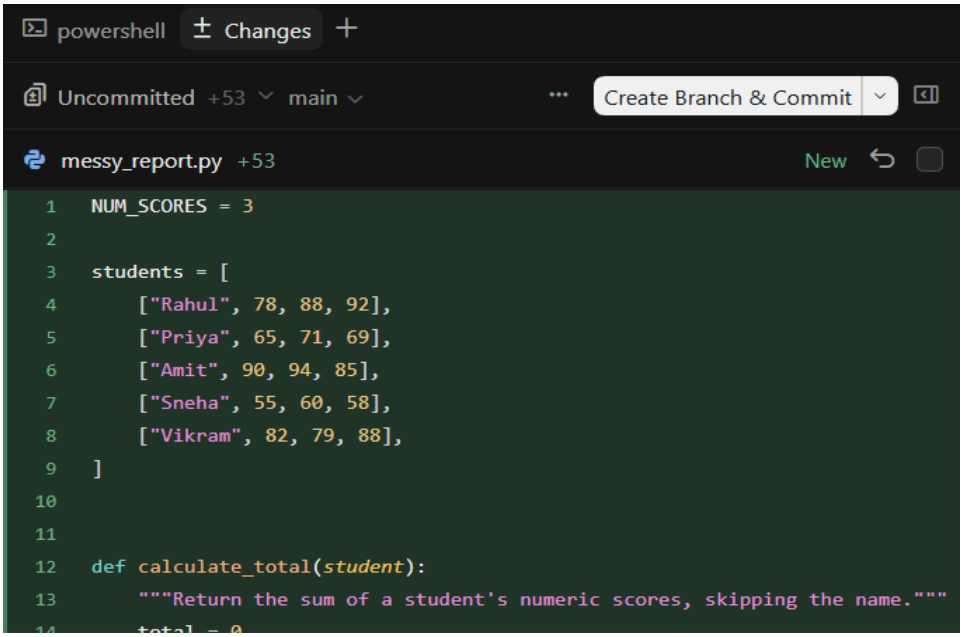
Refactor this script for readability. The printed output must stay exactly the same, character for character.

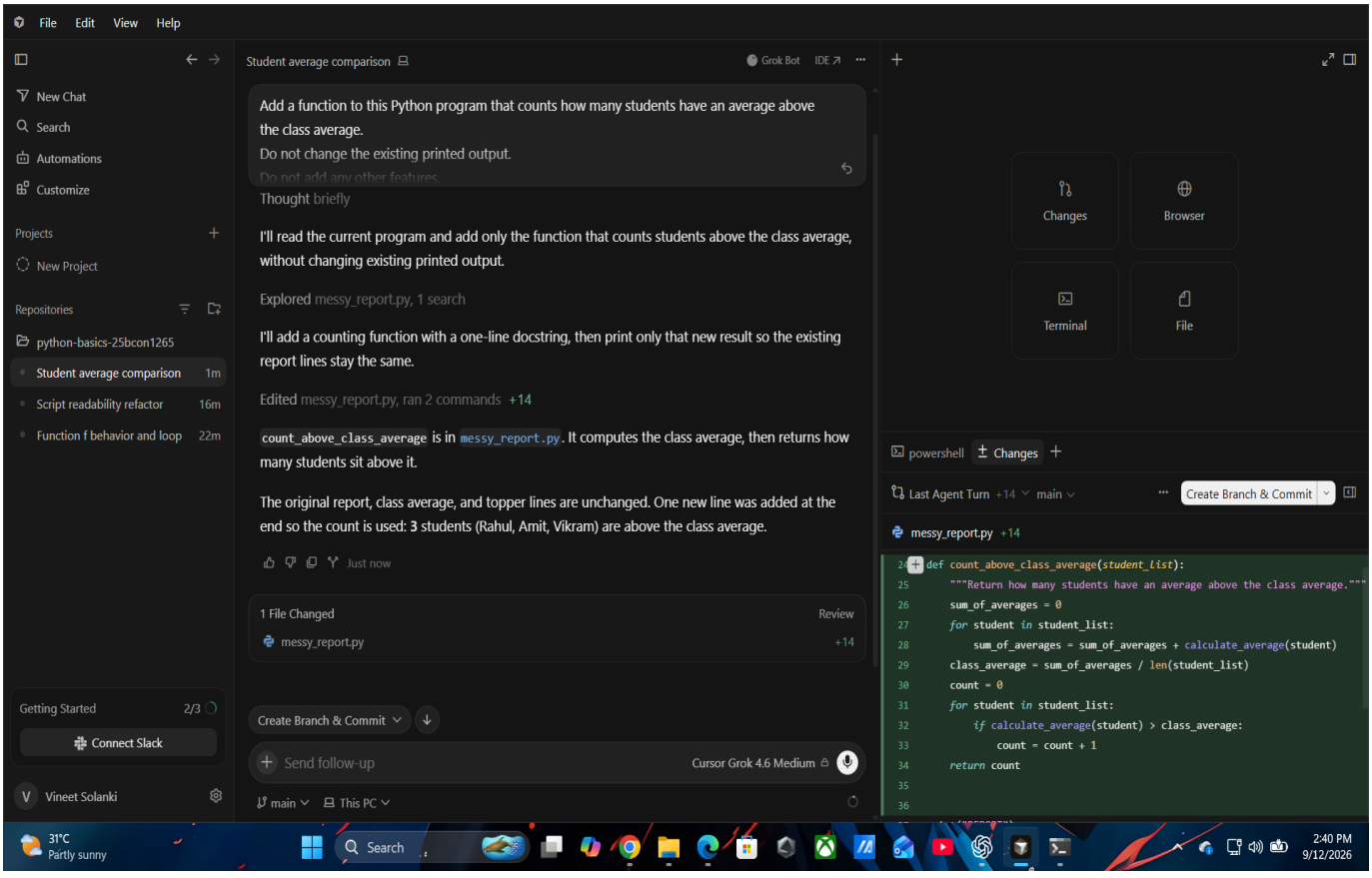
Specifically: use descriptive names, remove unused imports, replace the repeated average calculation with a single reusable function, name any hard-coded number that is used more than once, and use f-strings instead of concatenation.

Refactor Verification

Check	Result	Evidence
Read every proposed diff	Completed	Reviewed Cursor's proposed refactoring diff before accepting the changes.
Only explainable changes accepted	Completed	Accepted only the readability changes: removed unused math, renamed variables, added reusable calculate_average(), introduced NUM_SCORES, and replaced string concatenation with f-strings.
Refactored code runs	Passed	messy_report.py ran successfully after the refactoring.
Output matches original character-for-character	Passed	Compared the refactored output with original_output.txt; the output matched exactly.
Mismatch, if any, resolved and documented	No mismatch	No mismatch occurred, so no correction was required.




powershell
Changes
Uncommitted +53
main
Create Branch & Commit
messy_report.py +53
New
NUM_SCORES = 3
students = [
["Rahul", 78, 88, 92],
["Priya", 65, 71, 69],
["Amit", 90, 94, 85],
["Sneha", 55, 60, 58],
["Vikram", 82, 79, 88],
]
def calculate_total(student):
"""Return the sum of a student's numeric scores, skipping the name."""


File Edit View Help
Student average comparison
Grok Bot
IDE
New Chat
Search
Automations
Customize
Projects
New Project
Repositories
python-basics-25bcon1265
Student average comparison 1m
Script readability refactor 16m
Function f behavior and loop 22m
Getting Started 2/3
Connect Slack
Vineet Solanki
Add a function to this Python program that counts how many students have an average above the class average.
Do not change the existing printed output.
Do not add any other features
Thought briefly
I'll read the current program and add only the function that counts students above the class average, without changing existing printed output.
Explored messy_report.py, 1 search
I'll add a counting function with a one-line docstring, then print only that new result so the existing report lines stay the same.
Edited messy_report.py, ran 2 commands +14
count_above_class_average is in messy_report.py. It computes the class average, then returns how many students sit above it.
The original report, class average, and topper lines are unchanged. One new line was added at the end so the count is used: 3 students (Rahul, Amit, Vikram) are above the class average.
Just now
1 File Changed
Review
messy_report.py +14
Create Branch & Commit
Send follow-up
Cursor Grok 4.6 Medium
main
This PC
Changes
Browser
Terminal
File
powershell
Changes
Last Agent Turn +14
main
Create Branch & Commit
messy_report.py +14
def count_above_class_average(student_list):
"""Return how many students have an average above the class average."""
sum_of_averages = 0
for student in student_list:
sum_of_averages = sum_of_averages + calculate_average(student)
class_average = sum_of_averages / len(student_list)
count = 0
for student in student_list:
if calculate_average(student) > class_average:
count = count + 1
return count
31°C
Partly sunny
Search
2:40 PM
9/12/2026

3. Activity C — Project Rules File

Rules File Content

```
---

description: Python coding standards for this course

globs: ["**/*.py"]

alwaysApply: true

---


- Use descriptive variable names. Single letters are allowed only for loop counters.

- Every function has a one-line docstring stating what it returns.

- Use f-strings for output, never string concatenation.

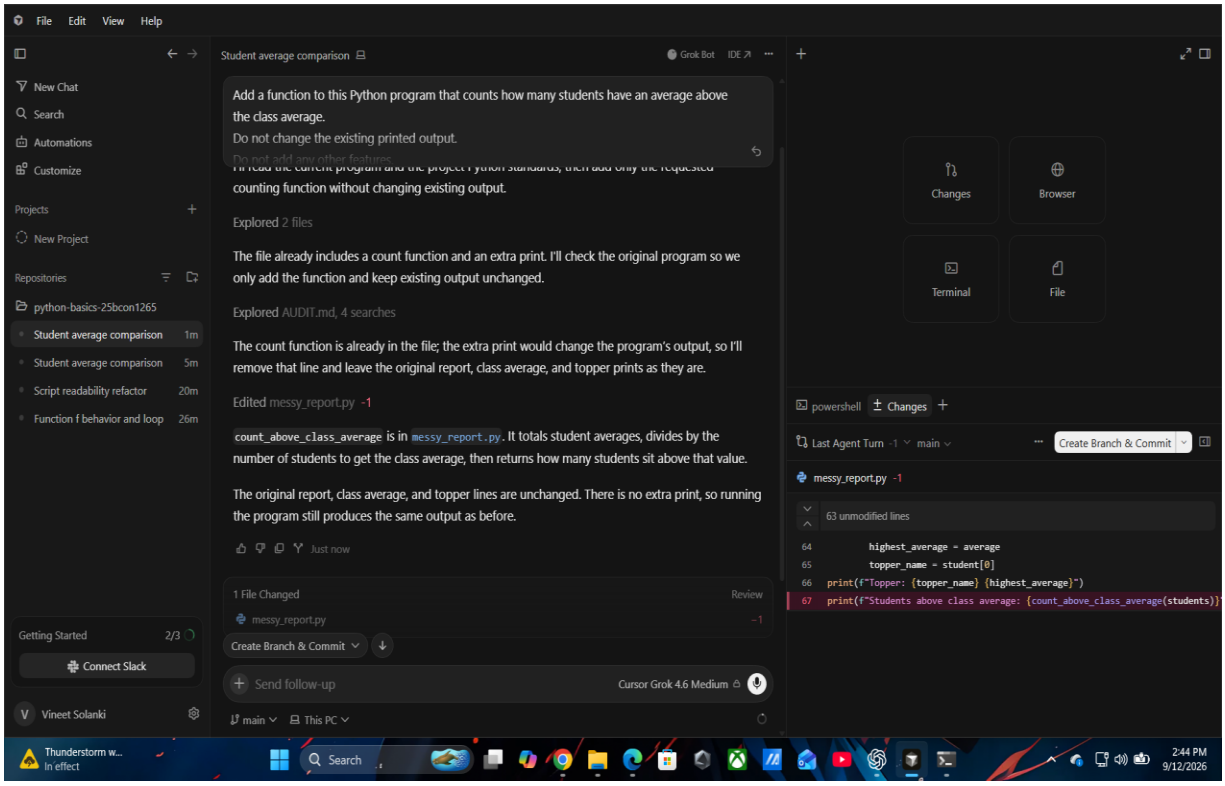
- Do not hard-code a number that is used more than once. Name it.

- Do not add imports that are not used.

- Do not add features that were not requested.

- Use snake_case naming for functions and variables.

- Keep each function focused on one clear responsibility.
```

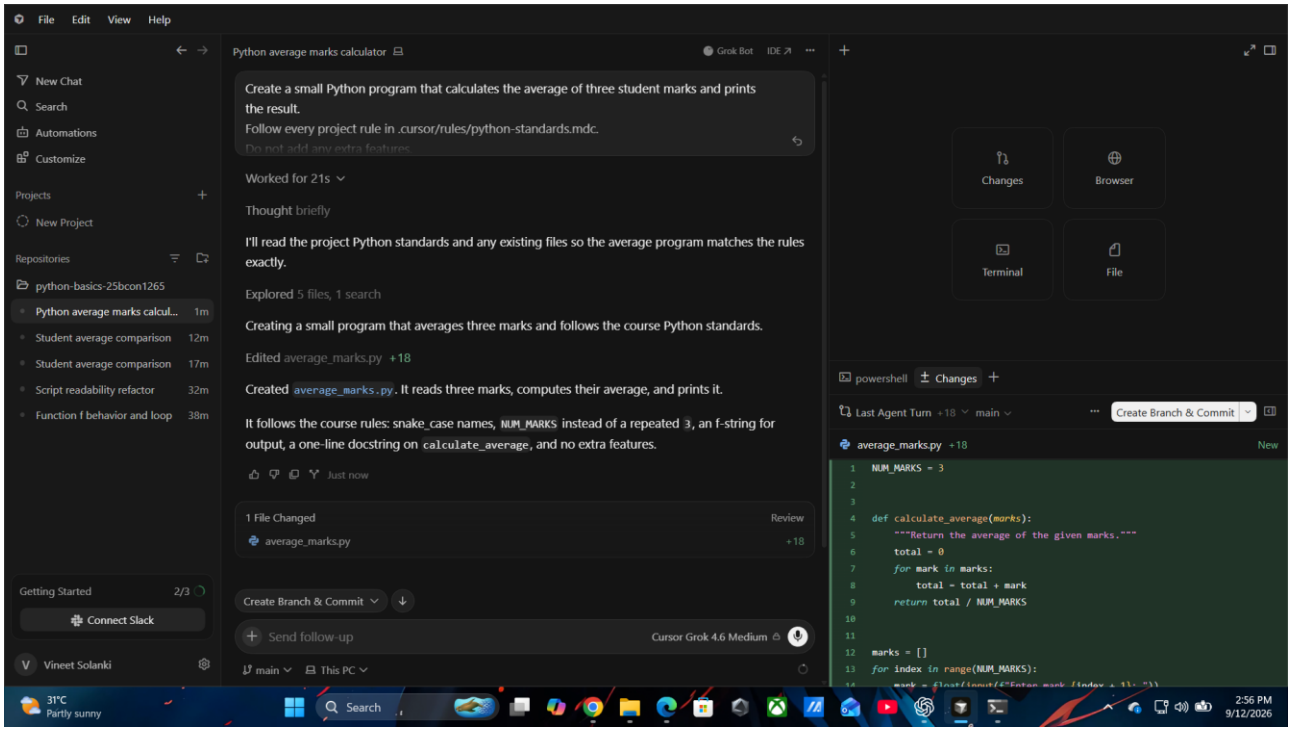


4. Six-Row Rules Comparison

Rule	Followed?	Evidence from generated code
Descriptive variable names	YES	def calculate_average(marks): and average = sum(marks) / NUM_MARKS use descriptive names.
One-line function docstring	YES	"""Calculate and return the average of the marks."""
f-strings, not concatenation	YES	print(f"Average: {average}")
No unnamed repeated numbers	YES	NUM_MARKS = 3 is used instead of repeatedly writing 3.
No unused imports	YES	No unnecessary import statements are present in the generated code.
No unrequested features	YES	The program only calculates the average of three marks and prints the result; no extra features were added.

AI Mode Suitability — One Line Each

AI mode	What it is good for
Understand-only	Good for unfamiliar code, explanations, and planning without edits.
Edit in place	Good for a small one-file change that can be described in one sentence.
Multi-file edit	Good for cross-file refactors, but every proposed diff must be reviewed.



5. Review Habits + Personal Reflection

Personal Reflection

I learned that AI-generated code should not be accepted without reviewing the changes. I will first understand what the code does, then check the diff and verify that the output has not changed. Project rules are useful because they make coding standards consistent and help me notice issues such as unclear names, repeated numbers, unused imports, and unnecessary features.

Commit Order

Order	Expected repository evidence
1.	original_output.txt committed before refactoring.
2.	Refactored script committed with output verification.
3.	Note stating exact match or explaining mismatch.
4.	.cursor/rules/python-standards.mdc committed.
5.	Six-row rules comparison with evidence.

6. Final Submission Checklist

- ☐ Two HW-04 program outputs captured before refactoring.
- ☐ original_output.txt committed before refactoring.
- ☐ Refactored script committed with output verification.
- ☐ Note stating exact match or explaining any mismatch.
- ☐ .cursor/rules/python-standards.mdc created and committed.
- ☐ Six-row rules comparison completed with actual code evidence.
- ☐ One line recorded for what each AI mode is good for.
- ☐ Personal review-habit reflection written in my own words.
- ☐ Repository link included in the final PDF.
- ☐ PDF filename: HW06_25BCON1265_Vineet_Solanki.pdf.

Repository Details

Field	Details
Repository	python-basics-25BCON1265
Visibility	Public
Repository link	https://github.com/Vineet07-tech/python-basics-25BCON1265.git